

Universidad Nacional de Colombia

Introducción a las Ciencias de la Computación

Semestre: 2026-1

LogicCheck: un pequeño verificador de satisfacibilidad.

Docente: Carlos Manuel Orrego Franco

Sección/Grupo: Grupo: 7

Fecha de entrega: 19 de junio de 2026

Integrantes:

Mariana Zapata Fernández

Alaham Daniel Aarón Daza

Andres Fabricio Pizo

Resumen

Este informe desarrolla la evaluación de logicheck Lab, una herramienta educativa para mostrar cómo una computadora puede razonar sobre fórmulas proposicionales pequeñas. Donde se busca construir un verificador de satisfacibilidad proposicional (SAT) que determine si una fórmula lógica dada es satisfacible o no. Para esto se implementa un algoritmo de backtracking que explora el espacio de asignaciones posibles para las variables de la fórmula, aplicando técnicas de poda para descartar ramas del árbol de búsqueda que no pueden conducir a una solución satisfacible. El informe incluye una descripción detallada del procedimiento seguido, un modelamiento manual de un caso pequeño para ilustrar el funcionamiento del algoritmo, y una implementación en Python. Además, se discuten las decisiones tomadas durante el desarrollo, los resultados obtenidos y las conclusiones finales sobre la efectividad del enfoque utilizado.

1. Introducción

La satisfacibilidad proposicional (SAT) es un problema fundamental en la informática teórica y práctica, que consiste en determinar si existe una asignación de valores de verdad a las variables de una fórmula lógica que haga que la fórmula sea verdadera. Este problema es de gran importancia porque es el primer problema que se demostró ser NP-completo, lo que implica que es tan difícil como cualquier otro problema en la clase NP. La capacidad de resolver SAT de manera eficiente tiene aplicaciones en áreas como la verificación de software, la inteligencia artificial, la optimización y la criptografía. Sin embargo, a medida que el número de variables en una fórmula aumenta, el número de combinaciones posibles crece exponencialmente, lo que hace que una evaluación exhaustiva (fuerza bruta) sea inviable para fórmulas con muchas variables. En este contexto, el enfoque de backtracking se presenta como una estrategia efectiva para explorar el espacio de soluciones de manera más inteligente, evitando la evaluación de combinaciones que ya se han determinado como no satisfacibles. Este informe se centrará en la implementación de un verificador de SAT utilizando backtracking, incluyendo un ejemplo manual para ilustrar el proceso, una implementación en Python y una discusión detallada de los resultados obtenidos.

2. Descripción del problema.

El problema que se plantea es que al haber mayor cantidad de variables en una fórmula lógica, el número de combinaciones posibles para evaluar su satisfacibilidad crece exponencialmente, lo que hace inviable la generación de tablas de verdad completas. Para solucionar este problema, se propone modelar el sistema de Backtracking (búsqueda con

retroceso), que permite explorar el espacio de soluciones de manera más eficiente al evitar la evaluación de combinaciones que ya se han determinado como no satisfacibles.

3. Modelamiento con backtracking.

Para resolver el problema de la satisfacibilidad proposicional (SAT) de manera más eficiente que una exploración por fuerza bruta (tablas de verdad), se modeló el sistema utilizando el paradigma de Backtracking (búsqueda con retroceso). Este enfoque modela el espacio de soluciones como un árbol de decisión binario, donde cada nivel del árbol representa una variable proposicional y cada rama representa la asignación de un valor de verdad (*True* o *False*).

A continuación, se definen formalmente los componentes del algoritmo aplicados al sistema LogicCheck:

Solución Parcial: Es un estado intermedio de la búsqueda representado por un diccionario o conjunto de pares clave-valor $\mathcal{A} = \{v_1 : valor_1, v_2 : valor_2, \dots\}$, donde solo un subconjunto de las variables de la fórmula ha recibido un valor de verdad definitivo.

Candidatos: Para la siguiente variable libre (sin asignar) en el orden de evaluación, el conjunto de candidatos está estrictamente limitado a los valores booleanos: $C = \{True, False\}$.

Restricciones (Criterio de Poda): Es la condición lógica que evalúa la viabilidad de la solución parcial. Una cláusula se define como definitivamente falsa si y solo si todos los literales que la componen han sido evaluados bajo la asignación parcial actual y el resultado de su disyunción ($\vee$) es falso. Cuando el sistema detecta una cláusula definitivamente falsa, se activa el mecanismo de poda (pruning), deteniendo la exploración de esa rama de inmediato.

Solución Completa: Se alcanza cuando el tamaño de la asignación parcial es igual al número total de variables únicas en la fórmula ($|\mathcal{A}| = |Variables|$) y, al evaluar la conjunción ($\wedge$) de todas las cláusulas, ninguna resulta ser falsa.

Retroceso(Backtrack): Si al explorar un candidato para la variable actual se genera una violación de las restricciones (una cláusula definitivamente falsa), o si la recursión subsecuente retorna que no hay soluciones en esa rama, el algoritmo deshace la última asignación en el diccionario (limpieza del estado) y regresa al nivel anterior para probar con el candidato alternativo o retroceder más arriba en el árbol.

Formalización del Árbol de Búsqueda y Eficiencia diferencia de un método iterativo que genera de forma ciega las 2^n combinaciones posibles (donde n es el número de variables), el algoritmo con backtracking evalúa de forma perezosa (lazy evaluation) las cláusulas. Si una contradicción ocurre en una variable temprana (por ejemplo, en el nivel 2 del árbol), el algoritmo descarta de golpe todas las combinaciones que se derivarían de las

variables restantes. Esto transforma el tiempo de ejecución en el caso promedio, reduciendo significativamente el número de nodos visitados en comparación con la evaluación de una tabla de verdad completa.

4. Ejemplo manual pequeño.

Caso de Estudio: El Enigma del Turno de Fin de Semana Historia (Descripción del Problema).

Una empresa de software necesita organizar el turno de soporte técnico para este fin de semana. Tienen a tres programadores disponibles: Ana (A), Beto (B) y Carlos (C). Recursos Humanos ha establecido las siguientes reglas lógicas para asignar los turnos:

- Regla 1: Al menos uno de los tres debe trabajar en el turno.
- Regla 2: Ana y Beto no se llevan bien, por lo que no pueden trabajar juntos.
- Regla 3: Carlos es un programador Junior y necesita supervisión. Si Carlos trabaja, Ana debe trabajar obligatoriamente para supervisarlos.
- Regla 4: Por ser el líder del proyecto este mes, Beto debe trabajar obligatoriamente.

Modelamiento del Problema:

- A : Ana trabaja (True o False)
- B : Beto trabaja (True o False)
- C : Carlos trabaja (True o False)

Transformamos las reglas a formato de lista de cláusulas (donde cada lista interna es una disyunción).

- Regla 1: $[A, B, C]$ (A o B o C)
- Regla 2: $[A, B]$ (No Ana o No Beto)
- Regla 3: $[C, A]$ (No Carlos o Ana)
- Regla 4: $[B]$ (Beto obligatoriamente)

Fórmula Final en el Código: $\text{Fórmula} = [[A, B, C], [A, B], [C, A], [B]]$

5. Implementación en python.

Listing 1: Implementación de Backtracking para SAT

```
def get_variables(formula):
    """Extrae todas las variables únicas de una fórmula lógica."""
    vars_set = set()
    for clause in formula:
        for literal in clause:
            vars_set.add(literal.replace("-", ""))
```

```
    return sorted(list(vars_set))

def evaluate_clause(clause, assignment):
    """
    Evalúa si una cláusula es verdadera, falsa o incierta.
    Retorna True si es definitivamente verdadera,
    False si es definitivamente falsa,
    None si no se puede determinar aún.
    """
    is_false = True
    for literal in clause:
        is_neg = literal.startswith("-")
        var = literal.replace("-", "")
        val = assignment.get(var)

        if val is None:
            is_false = False # Falta información, no es
                             # definitivamente falsa
        else:
            # Si un literal es True, toda la cláusula es True (
            # disyunción)
            if (val and not is_neg) or (not val and is_neg):
                return True

    if is_false:
        return False # Todos los literales evaluados fueron
                     # falsos

    return None

def solve_sat(formula, variables, index, assignment, stats):
    stats["llamadas_recursivas"] += 1

    # RESTRICCIONES (Poda)
    for clause in formula:
        if evaluate_clause(clause, assignment) is False:
            stats["ramas_podadas"] += 1
            return False

    # CASO BASE: Solución Completa
    if index == len(variables):
```

```
        return True

    var = variables[index]

    # CANDIDATOS
    for candidate in [True, False]:
        assignment[var] = candidate

        if solve_sat(formula, variables, index + 1, assignment,
            stats):
            return True

    # RETROCESO (Backtrack)
    stats["retrocesos"] += 1
    assignment[var] = None

    return False

def run_logic_check(formula):
    variables = get_variables(formula)
    assignment = {v: None for v in variables}
    stats = {"llamadas_recurativas": 0, "ramas_podadas": 0, "
        retrocesos": 0}

    is_satisfiable = solve_sat(formula, variables, 0, assignment,
        stats)

    if is_satisfiable:
        return True, {k: v for k, v in assignment.items() if v is
            not None}, stats
    else:
        return False, None, stats
```

6. Pruebas y resultados.

Dado pruebas con las formulas sugeridas en el enunciado, del problema

Listing 2: Pruebas con fórmulas de ejemplo

```
casos_prueba = [
    # 3 Fórmulas Satisfacibles
    ("Caso SAT 1 (Ejemplo Manual)", [[ "-A", "-B"], ["A", "B"] ]),
```

```

("Caso SAT 2 (Implicación)", [[ "-A", "B"], ["A"] ]),
("Caso SAT 3 (Múltiples Variables)", [[ "A", "B", "-C"], ["-A",
    , "B"], ["C"] ]),

# 2 Fórmulas Insatisfacibles
("Caso UNSAT 1 (Contradicción Directa)", [[ "A"], ["-A"] ]),
("Caso UNSAT 2 (Ciclo Imposible)", [[ "A", "B"], ["-A"], ["-B"
    ] ] )

]

for nombre, formula in casos_prueba:
    print(f"--- {nombre} ---")
    print(f"Fórmula: {formula}")
    satisfiable, asignacion, stats = run_logic_check(formula)

    if satisfiable:
        print(f"Resultado: SATISFACIBLE \nAsignación: {asignacion
            }")
    else:
        print("Resultado: INSATISFACIBLE (No existe combinación v
            álida)")

    print(f"Estadísticas: {stats}\n")

```

Resultados Observados:

Listing 3: Resultados observados

```

--- Caso SAT 1 (Ejemplo Manual) ---
Fórmula: [['-A', '-B'], ['A', 'B']]
Resultado: SATISFACIBLE
Asignación: {'A': True, 'B': False}
Estadísticas: {'llamadas_recurativas': 3, 'ramas_podadas': 0, '
    retrocesos': 0}

--- Caso SAT 2 (Implicación) ---
Fórmula: [['-A', 'B'], ['A']]
Resultado: SATISFACIBLE
Asignación: {'A': True, 'B': True}
Estadísticas: {'llamadas_recurativas': 2, 'ramas_podadas': 0, '
    retrocesos': 0}

--- Caso SAT 3 (Múltiples Variables) ---

```

```
Fórmula: [['A', 'B', '-C'], ['-A', 'B'], ['C']]
Resultado: SATISFACIBLE
Asignación: {'A': False, 'B': True, 'C': True}
Estadísticas: {'llamadas_recursivas': 4, 'ramas_podadas': 0, 'retrocesos': 0}

--- Caso UNSAT 1 (Contradicción Directa) ---
Fórmula: [['A'], ['-A']]
Resultado: INSATISFACIBLE (No existe combinación válida)
Estadísticas: {'llamadas_recursivas': 2, 'ramas_podadas': 0, 'retrocesos': 0}

--- Caso UNSAT 2 (Ciclo Imposible) ---
Fórmula: [['A', 'B'], ['-A'], ['-B']]
Resultado: INSATISFACIBLE (No existe combinación válida)
Estadísticas: {'llamadas_recursivas': 3, 'ramas_podadas': 0, 'retrocesos': 0}
```

7. Decisiones del grupo

- ¿Qué lenguaje de programación usaron y por qué? Usamos Python, porque permite modelar rápidamente estructuras de datos como listas y diccionarios, y su legibilidad facilita implementar recursividad de forma intuitiva, lo que es ideal para Backtracking.
- ¿Qué representación eligieron y por qué? Representamos la fórmula como una lista de listas de cadenas [['A', 'B'], ['-A']], y la solución parcial como un diccionario (".A": True, "B": None). Esto nos permite evaluar rápida y fácilmente el estado de cualquier variable en $O(1)$.
- ¿Qué restricciones implementaron primero? La restricción crítica de que ninguna cláusula puede ser definitivamente falsa bajo la asignación parcial actual. Si una sola cláusula falla irremediablemente, la fórmula entera falla.
- ¿Qué bug encontraron durante el desarrollo? Inicialmente no distinguíamos entre una cláusula con valor "Falso definitivo" y una cláusula ".Aún no evaluada completamente". Esto provocaba que podáramos ramas prematuramente. Lo solucionamos implementando un retorno tri-estado (True, False, None) en evaluate clause.
- ¿Qué caso de prueba falló inicialmente? El caso de fórmulas insatisfacibles simples como [".A"], [A]]. El algoritmo no lograba retroceder correctamente al principio porque las variables no se re-asignaban a None al realizar el Backtrack.
- ¿Qué poda implementaron? Podamos la rama en el mismo instante en que se de-

tecta que cualquier cláusula de la fórmula se evalúa como definitivamente falsa. De esta forma, evitamos explorar el resto de variables que no cambiarían el hecho de que esa cláusula falló. Se descarta inmediatamente cualquier candidato que viole las restricciones (una cláusula que no cuadra o un número repetido), ahorrando la exploración de millones de ramas inútiles.

- ¿Qué cambiarían si tuvieran más tiempo? Implementaríamos una heurística de ordenamiento de variables para evaluar primero las variables que aparecen en más cláusulas o en cláusulas unitarias (como en el algoritmo DPLL), lo cual aumentaría la cantidad de podas tempranas y aceleraría el tiempo de ejecución.
- ¿Qué parte hizo cada integrante? Mariana: Diseño general del algoritmo, modelamiento y visualización de la traza paso a paso, Alaham: Implementación de la función `evaluate clause` y el chequeo de restricciones lógicas. Andres: Creación de los casos de prueba, ejecución de las estadísticas y ensamblado final del notebook.

8. Conclusiones

- Se logró diseñar e implementar un verificador de satisfacibilidad proposicional utilizando el modelo de backtracking, lo que permitió resolver el problema de manera eficiente al evitar la generación de todas las combinaciones posibles. Observando las estadísticas en nuestras pruebas, es evidente cómo el mecanismo de retroceso y la evaluación temprana de restricciones descartan ramas inviables significativamente, previniendo así un decaimiento drástico del rendimiento (lo que ocurriría en un enfoque de fuerza bruta completa).
- La implementación en Python facilitó la manipulación de estructuras de datos dinámicas como listas y diccionarios, lo que fue crucial para mantener el estado de las asignaciones parciales y evaluar las cláusulas de manera eficiente. Además, la claridad del código permitió una fácil identificación y corrección de bugs durante el desarrollo.

Referencias

- [1] S. Cook, “The complexity of theorem-proving procedures,” in *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 1971, pp. 151–158.
- [2] C. M. Orrego Franco, “Guía de laboratorio: LogicCheck Lab y verificación de satisfacibilidad,” Universidad Nacional de Colombia, Sede Manizales, *Introducción a las Ciencias de la Computación (Semestre 2026-1)*, 2026.

- [3] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.

9. Declaración de autoria y uso de herramientas

Declaración de Autoría

Los estudiantes, **Mariana Zapata Fernández**, **Alaham Daniel Aarón Daza** y **Andres Fabricio Pizo**, regulares de la Universidad Nacional de Colombia en el curso de *Introducción a las Ciencias de la Computación* (Grupo 7, Semestre 2026-1), declaramos que:

1. Somos los autores intelectuales y materiales del presente informe y del código fuente adjunto denominado **LogicCheck**.
2. La distribución del trabajo se realizó de manera equitativa y colaborativa bajo los siguientes roles:
 - **Mariana Zapata Fernández:** Diseño general del algoritmo de backtracking, modelamiento y visualización de la traza paso a paso del árbol de decisión.
 - **Alaham Daniel Aarón Daza:** Implementación y lógica de la función de evaluación de cláusulas (`evaluate_clause`) y el chequeo de restricciones tri-estado.
 - **Andres Fabricio Pizo:** Creación de los casos de prueba, recolección de estadísticas de rendimiento y ensamblado final del reporte/notebook.

Uso de Herramientas de Inteligencia Artificial (IA)

En cumplimiento de las directrices académicas e institucionales sobre la transparencia en el uso de tecnologías emergentes, declaramos que para el desarrollo de este proyecto:

- **Herramientas utilizadas:** Se emplearon modelos de lenguaje (LLMs) como asistentes de programación y redacción.
- **Alcance del uso:**
 - Se utilizaron como apoyo para identificar errores de sintaxis en el entorno de desarrollo y estructurar la lógica del retorno tri-estado (`True`, `False`, `None`) tras detectar fallos en el retroceso manual.
 - Se emplearon para la revisión ortográfica, corrección de estilo y la organización formal de este documento de texto.